
Gradient Descent for Machine Learning

From intuitive hill-walking to backpropagation and deep neural network training

All mathematical equations are typeset as native LaTeX in the source and PDF; equation screenshots are not used.

This report is written for someone who wants to become genuinely comfortable with modern machine learning and deep learning. It treats gradient descent as the central training mechanism that connects a model's parameters, a loss function, calculus, backpropagation, and practical optimization.

The single sentence to remember: a machine-learning model makes predictions; the loss function measures how wrong they are; the gradient tells us how the loss changes when each parameter changes; gradient descent uses that information to move the parameters toward lower loss.

Learning goals By the end, you should be able to read equations such as

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} J(\theta_t),$$

understand every symbol, explain why the minus sign is there, distinguish a gradient from a loss, explain how backpropagation obtains gradients, and understand where SGD, momentum, RMSProp, and Adam fit into deep-learning training.

Contents

1	Why Gradient Descent Matters in Machine Learning	6
1.1	What is actually being optimized?	6
1.2	Why not just calculate the best parameters directly?	6
1.3	What gradient descent does not guarantee	7
2	The Machine-Learning Objective: Data, Parameters, and Loss	7
2.1	Parameters and predictions	7
2.2	Loss functions	7
2.3	Empirical risk and population risk	8
2.4	Why the loss is the bridge between learning and optimization	8
3	From Derivatives to the Gradient	8
3.1	The derivative in one dimension	8
3.2	Many parameters: one slope is no longer enough	9
3.3	What the components mean	9
3.4	Why the gradient points uphill	9
4	The Core Algorithm	10
4.1	The update rule	10
4.2	A one-dimensional numerical example	10
4.3	Gradient descent as repeated local approximation	11
5	The Geometry of Learning	12
5.1	Orthogonality to contours	12
5.2	Why paths can zig-zag	13
5.3	Parameters as coordinates of a huge space	13
6	Learning Rate: The Most Important Hyperparameter to Understand	13
6.1	Too small	14
6.2	Reasonable	14
6.3	Too large	14
6.4	A precise one-dimensional stability result	14
6.5	Learning-rate schedules	15
7	Batch Gradient Descent, SGD, and Minibatches	15

7.1	Full-batch gradient descent	15
7.2	Stochastic gradient descent	15
7.3	Minibatch SGD	16
7.4	Epochs, iterations, and batch size	16
7.5	Why minibatch gradients are useful estimators	16
8	A Worked Machine-Learning Example: Linear Regression	17
8.1	Model	17
8.2	Mean squared error	17
8.3	Matrix form	17
8.4	Why feature scaling matters	18
9	A Second Worked Example: Logistic Regression and Cross-Entropy	18
9.1	Model	18
9.2	Cross-entropy loss	19
9.3	Interpretation	20
10	Second-Order Intuition: Curvature and Why the Landscape Matters	20
10.1	The Hessian	20
10.2	Why the Hessian is conceptually useful even when we do not compute it	20
10.3	Smoothness and a useful descent inequality	20
11	Momentum and Adaptive Optimization	21
11.1	Momentum	21
11.2	Nesterov momentum	22
11.3	AdaGrad	22
11.4	RMSProp	22
11.5	Adam	23
11.6	Do adaptive optimizers replace understanding learning rates?	23
12	Deep Learning: Where Gradient Descent Meets Backpropagation	24
12.1	A neural network is a parameterized computation graph	24
12.2	The difficult part is not the update rule	25
12.3	The chain rule	25
12.4	Backpropagation and gradient descent are different jobs	26
13	Deep Learning Training in More Detail	26

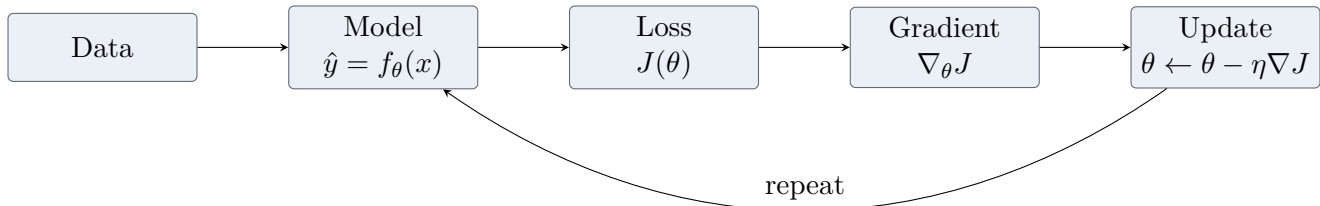
13.1 One minibatch, end to end	26
13.2 Why deep networks need nonlinearities	27
13.3 Vanishing and exploding gradients	27
13.4 Why stochasticity can help	27
13.5 Training loss versus validation loss	27
14 How Gradient Descent Appears Across Machine Learning	28
14.1 Linear regression	28
14.2 Logistic regression	28
14.3 Generalized linear models	28
14.4 Matrix factorization and recommender systems	28
14.5 Neural networks	28
14.6 Support vector machines	28
14.7 Gradient boosting	29
15 What Machine-Learning Methods Do Not Need Gradient Descent?	29
15.1 Why trees behave differently	29
16 Regularization, Weight Decay, and the Objective You Actually Optimize	30
16.1 L2 regularization	30
16.2 L1 regularization	30
16.3 The practical lesson	30
17 Convergence, Minima, Saddles, and Nonconvexity	30
17.1 Convex objectives	30
17.2 Neural networks are typically nonconvex	31
17.3 Why “gradient descent finds the global minimum” is the wrong generic statement	31
18 Optimization Pathologies in Deep Learning	31
18.1 Vanishing gradients	31
18.2 Exploding gradients	31
18.3 Ill-conditioning	31
18.4 Saturation	32
18.5 Dead units	32
18.6 Numerical issues	32
19 What the Training Curves Are Telling You	32

19.1	Checking gradient magnitudes	32
19.2	Why a low loss is not the entire story	33
20	A Practical Mental Model for Deep-Learning Optimizers	33
20.1	A useful interpretation of Adam	34
20.2	Why SGD is still important to understand	34
21	Pseudocode and the Actual Training Loop	34
21.1	Automatic differentiation	34
21.2	One training step in symbols	35
22	A Deeper View: Gradient Descent as Optimization in Parameter Space	35
22.1	Reparameterization and effective geometry	35
22.2	Why optimization and architecture interact	35
22.3	Why initialization matters	35
23	The Most Important Equations to Know	36
23.1	Objective	36
23.2	Gradient	36
23.3	Gradient descent	36
23.4	Minibatch SGD	36
23.5	Momentum	36
23.6	RMSProp	36
23.7	Adam	36
23.8	Regularized objective	37
23.9	Backpropagation idea	37
24	How to Study Gradient Descent So It Actually Sticks	37
24.1	The mental translation to use when reading papers or code	37
24.2	A compact conceptual map	38
25	Common Misconceptions	38
26	Final Synthesis	38
	References and Further Reading	39
A	Notation Cheat Sheet	40

1 Why Gradient Descent Matters in Machine Learning

If deep learning is the subject you want to understand rather than merely use, gradient descent is one of the few concepts worth learning all the way down to the mathematics. Neural networks contain potentially millions or billions of adjustable numbers. Training means finding values for those numbers that make the network perform well on data. The network does not have an obvious formula saying which value every weight should take. Instead, training is an iterative search.

The important mental model is a loop:



This separation is fundamental. **The model is not gradient descent.** A linear regression model, logistic regression model, multilayer perceptron, and convolutional network are different models. Gradient descent is an optimization method that can be used to train many of them.

1.1 What is actually being optimized?

Suppose a model has parameters collected into a vector

$$\theta = \begin{bmatrix} \theta_1 & \theta_2 & \cdots & \theta_p \end{bmatrix}^T.$$

For a training example (x_i, y_i) , the model produces a prediction

$$\hat{y}_i = f_{\theta}(x_i).$$

A loss function measures the discrepancy between prediction and target:

$$\ell_i = \ell(f_{\theta}(x_i), y_i).$$

The training objective aggregates losses over the dataset:

$$J(\theta) = \frac{1}{n} \sum_{i=1}^n \ell(f_{\theta}(x_i), y_i).$$

Gradient descent changes θ in order to reduce $J(\theta)$.

Imagine θ as the settings of a gigantic machine and $J(\theta)$ as a score where lower is better. Training repeatedly asks: *which direction should I nudge all of the settings to make the score smaller?* The gradient answers that question locally.

1.2 Why not just calculate the best parameters directly?

For some simple models, direct solutions exist. Ordinary least squares can be written as a matrix equation. But direct algebra becomes impractical or impossible for modern neural networks, especially when the loss landscape is nonlinear and contains millions of interacting parameters.

Gradient-based optimization trades a closed-form answer for a sequence of relatively cheap, local computations. That is why the same basic idea survives from simple linear models to very large neural networks.

1.3 What gradient descent does not guarantee

Gradient descent does not magically make a model correct. It solves an optimization problem presented to it. If the dataset is noisy, biased, too small, or poorly specified, the optimizer can still efficiently find a bad solution. If the model is underpowered, it may not be able to achieve low loss. If the loss function rewards the wrong behavior, gradient descent faithfully optimizes the wrong objective.

For machine learning, always keep three questions separate:

What model? What objective/loss? What optimizer?

A neural network trained with Adam and the same network trained with SGD are the same model but different optimization procedures.

2 The Machine-Learning Objective: Data, Parameters, and Loss

Before deriving gradient descent, it helps to formalize the training problem itself.

2.1 Parameters and predictions

A model is a parameterized function

$$f_{\theta} : \mathcal{X} \rightarrow \mathcal{Y}.$$

The parameters θ are the quantities training changes. In a neural network, they are mainly weights and biases. In linear regression they may be coefficients and an intercept. In logistic regression they are the coefficients that determine a class probability.

Given input x , prediction is

$$\hat{y} = f_{\theta}(x).$$

The parameter vector may contain millions of entries, but mathematically we can still treat it as one point in $\mathbb{R}^p$.

2.2 Loss functions

A loss function translates prediction quality into a scalar. Typical examples include mean squared error for regression and cross-entropy for classification.

For regression with $y \in \mathbb{R}$:

$$\ell(\hat{y}, y) = \frac{1}{2}(\hat{y} - y)^2.$$

The factor $1/2$ is convenient because it cancels the 2 produced when differentiating.

For binary classification with predicted probability $p \in (0, 1)$ and target $y \in \{0, 1\}$:

$$\ell(p, y) = -[y \log p + (1 - y) \log(1 - p)].$$

For K -class classification, if the model outputs probabilities $p_1, \dots, p_K$ and the target class is represented by one-hot values y_k , categorical cross-entropy is

$$\ell(\mathbf{p}, \mathbf{y}) = - \sum_{k=1}^K y_k \log p_k.$$

Empirical training loss:

$$J(\theta) = \frac{1}{n} \sum_{i=1}^n \ell(f_{\theta}(x_i), y_i).$$

2.3 Empirical risk and population risk

The finite training objective is often called empirical risk:

$$\hat{R}(\theta) = \frac{1}{n} \sum_{i=1}^n \ell(f_{\theta}(x_i), y_i).$$

Conceptually, the ideal objective is population risk:

$$R(\theta) = \mathbb{E}_{(X,Y) \sim \mathcal{D}} [\ell(f_{\theta}(X), Y)].$$

Training data only provides a finite approximation to this expectation. This distinction matters because a parameter vector can have very low training loss while generalizing poorly to unseen examples.

2.4 Why the loss is the bridge between learning and optimization

The optimizer never needs to understand what an image, sentence, waveform, or customer record means semantically. It only sees a scalar objective and its derivatives. The semantics are encoded indirectly in the data and the model architecture.

This is one reason the same optimizer can train wildly different systems. The optimizer sees a vector of numbers, a scalar loss, and derivative information regardless of whether those numbers came from pixels, audio samples, tokens, or tabular features.

3 From Derivatives to the Gradient

Gradient descent becomes intuitive once the one-dimensional derivative is completely clear.

3.1 The derivative in one dimension

For a scalar function $f(x)$, the derivative at x is

$$f'(x) = \lim_{h \rightarrow 0} \frac{f(x+h) - f(x)}{h}.$$

It measures the local slope. If $f'(x) > 0$, increasing x slightly increases f . If $f'(x) < 0$, increasing x slightly decreases f .

Definition of the derivative:

$$f'(\theta) = \lim_{h \rightarrow 0} \frac{f(\theta+h) - f(\theta)}{h}.$$

The derivative is the local slope. If $f'(\theta) > 0$, increasing θ slightly increases the function; if $f'(\theta) < 0$, increasing θ slightly decreases it.

Suppose

$$f(x) = (x - 3)^2.$$

Then

$$f'(x) = 2(x - 3).$$

At $x = 5$, the derivative is 4. Moving to the right raises the function, so to reduce it we should move left.

3.2 Many parameters: one slope is no longer enough

Machine-learning models usually have many parameters:

$$J(\theta_1, \theta_2, \dots, \theta_p).$$

For each parameter we can compute a partial derivative:

$$\frac{\partial J}{\partial \theta_1}, \quad \frac{\partial J}{\partial \theta_2}, \quad \dots, \quad \frac{\partial J}{\partial \theta_p}.$$

Collect these into the gradient:

$$\nabla_{\theta} J(\theta) = \begin{bmatrix} \frac{\partial J}{\partial \theta_1} \\ \frac{\partial J}{\partial \theta_2} \\ \vdots \\ \frac{\partial J}{\partial \theta_p} \end{bmatrix}.$$

The gradient is therefore a vector, not a single number.

3.3 What the components mean

Each component answers a specific question: *if I increase this parameter by a tiny amount while holding the others fixed, what happens to the loss?*

If

$$\frac{\partial J}{\partial \theta_j} = +8,$$

then increasing θ_j tends to increase the loss locally. To reduce the loss, a sensible local move is to decrease θ_j .

If

$$\frac{\partial J}{\partial \theta_j} = -0.2,$$

then increasing θ_j tends to reduce the loss locally. To reduce the loss, we should increase θ_j .

The gradient collects all partial derivatives:

$$\nabla J(\theta) = \begin{bmatrix} \frac{\partial J}{\partial \theta_1} \\ \vdots \\ \frac{\partial J}{\partial \theta_d} \end{bmatrix}.$$

3.4 Why the gradient points uphill

A key theorem from multivariable calculus is that the gradient points in the direction of steepest local increase. Therefore $-\nabla J$ points in the direction of steepest local decrease.

Let $\mathbf{d}$ be any unit direction, $\|\mathbf{d}\| = 1$. The directional derivative is

$$D_{\mathbf{d}}J(\theta) = \nabla J(\theta)^\top \mathbf{d}.$$

By the Cauchy-Schwarz inequality,

$$\nabla J(\theta)^\top \mathbf{d} \geq -\|\nabla J(\theta)\|,$$

with the minimum achieved by

$$\mathbf{d} = -\frac{\nabla J(\theta)}{\|\nabla J(\theta)\|}.$$

That is the precise mathematical reason for the negative gradient.

4 The Core Algorithm

Now the familiar update rule can be derived rather than memorized.

4.1 The update rule

Let $\eta > 0$ be the learning rate. Gradient descent updates parameters according to

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} J(\theta_t).$$

Equivalently, parameter by parameter,

$$\theta_{j,t+1} = \theta_{j,t} - \eta \frac{\partial J}{\partial \theta_j}(\theta_t).$$

The four pieces have distinct meanings:

- θ_t : current parameter values.
- $\nabla J(\theta_t)$: local direction of greatest increase in loss.
- $-\nabla J(\theta_t)$: local direction of greatest decrease.
- η : how far to move in that direction.

Canonical gradient-descent update:

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t).$$

4.2 A one-dimensional numerical example

Take

$$f(w) = (w - 4)^2$$

and suppose we start at $w_0 = 0$. Since

$$f'(w) = 2(w - 4),$$

we have $f'(0) = -8$.

Choose $\eta = 0.25$. Then

$$w_1 = 0 - 0.25(-8) = 2.$$

At $w = 2$,

$$f'(2) = -4, \quad w_2 = 2 - 0.25(-4) = 3.$$

At $w = 3$,

$$f'(3) = -2, \quad w_3 = 3 - 0.25(-2) = 3.5.$$

The points approach the minimum at $w = 4$.

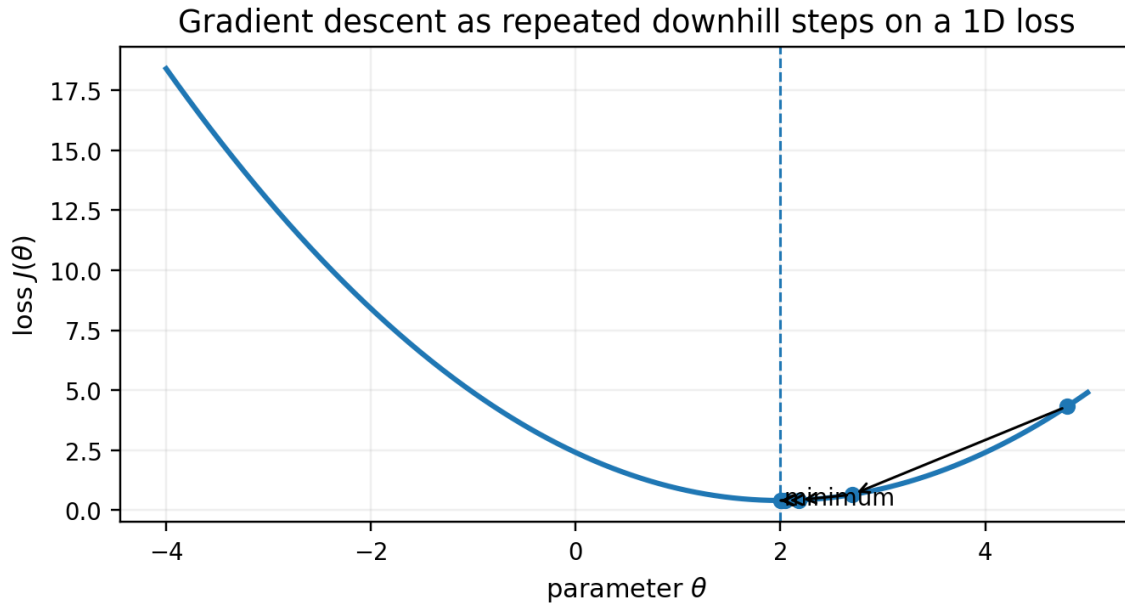


Figure 1: A one-dimensional optimization path: each update uses the local slope to move toward lower loss.

4.3 Gradient descent as repeated local approximation

At θ , consider a small step Δ . A first-order Taylor approximation gives

$$J(\theta + \Delta) \approx J(\theta) + \nabla J(\theta)^\top \Delta.$$

Set

$$\Delta = -\eta \nabla J(\theta).$$

Then

$$J(\theta - \eta \nabla J(\theta)) \approx J(\theta) - \eta \|\nabla J(\theta)\|^2.$$

Since the squared norm is nonnegative, the first-order prediction is a decrease whenever $\nabla J(\theta) \neq 0$ and $\eta > 0$.

First-order Taylor approximation:

$$J(\theta + \Delta) \approx J(\theta) + \nabla J(\theta)^\top \Delta.$$

With $\Delta = -\eta \nabla J(\theta)$,

$$J(\theta + \Delta) \approx J(\theta) - \eta \|\nabla J(\theta)\|^2.$$

The approximation explains both the power and the limitation of gradient descent. It is locally informed. It does not know the entire landscape in advance.

5 The Geometry of Learning

With two parameters, we can visualize a loss surface. Each point (θ_1, θ_2) corresponds to one model configuration and therefore one loss value.

A three-dimensional surface is intuitive, but contour plots are often more useful. A contour line contains parameter settings with equal loss. The center of nested contours represents a lower-loss region.

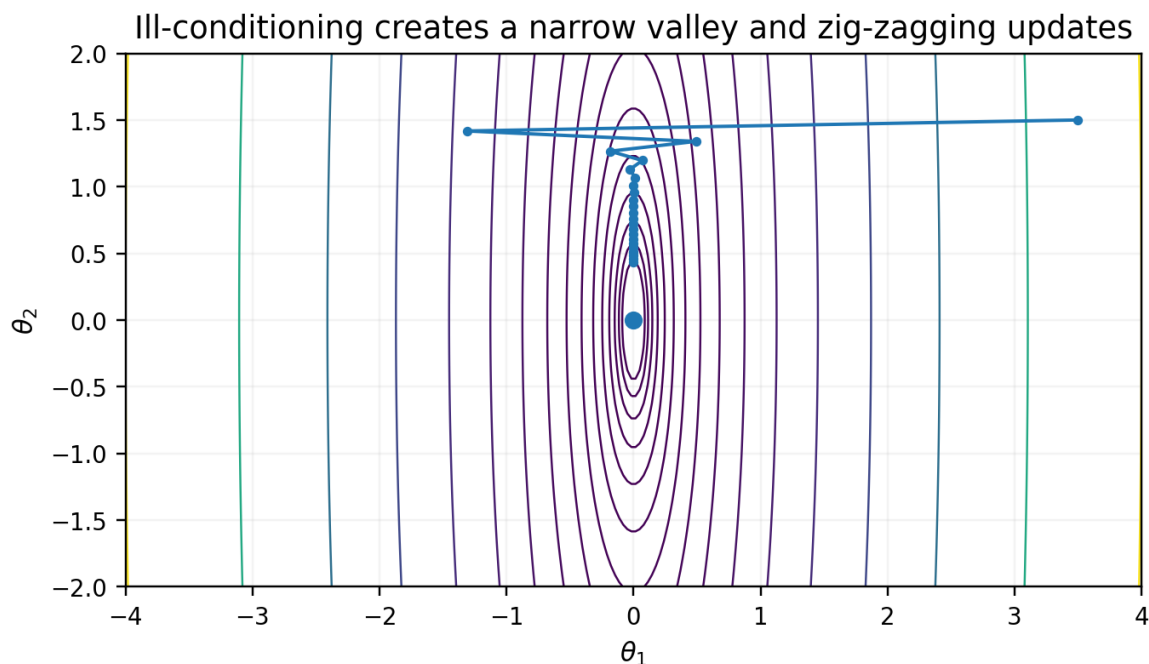


Figure 2: Gradient descent on a two-parameter loss surface. The trajectory follows local negative-gradient directions toward a low-loss point.

5.1 Orthogonality to contours

For a contour defined by $J(\theta_1, \theta_2) = c$, the gradient is perpendicular to the contour. Therefore a gradient step crosses contours rather than moving along them.

This provides a useful picture: the algorithm repeatedly asks, *which way is uphill right here?* and then walks in the opposite direction.

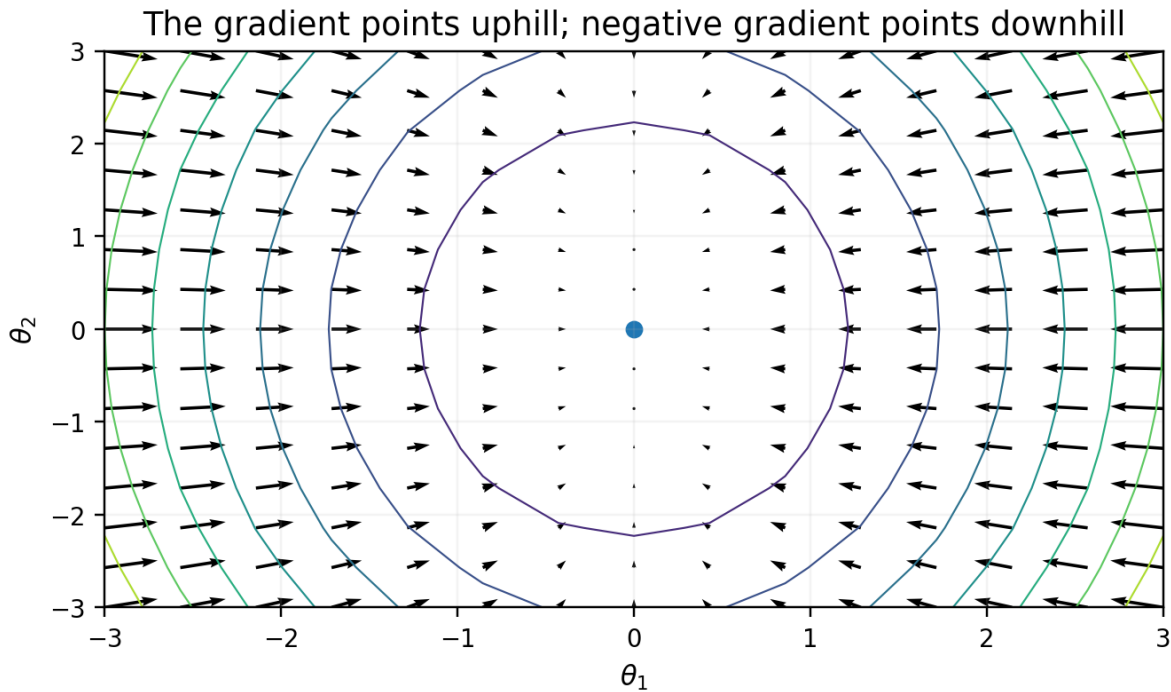


Figure 3: A gradient field visualizes the local direction information available to the optimizer.

5.2 Why paths can zig-zag

Suppose the loss has one steep direction and one shallow direction. A step large enough to make progress along the shallow direction can overshoot across the steep direction. The result is a zig-zag path.

This is strongly connected to **conditioning**, discussed later. Good optimization is not merely about having the correct direction; the scale and geometry of the problem determine whether that direction produces efficient progress.

5.3 Parameters as coordinates of a huge space

The two-dimensional picture is only a visualization. A network with ten million parameters lives in a ten-million-dimensional parameter space. We cannot draw that space, but the mathematics is the same:

$$\theta \in \mathbb{R}^{10^7}, \quad \nabla J(\theta) \in \mathbb{R}^{10^7}.$$

The optimizer manipulates vectors in this high-dimensional space without requiring us to visualize every coordinate.

6 Learning Rate: The Most Important Hyperparameter to Understand

The learning rate η controls step size. It is tempting to think that a larger learning rate is simply faster, but that is incomplete. A learning rate that is too large can make optimization unstable.

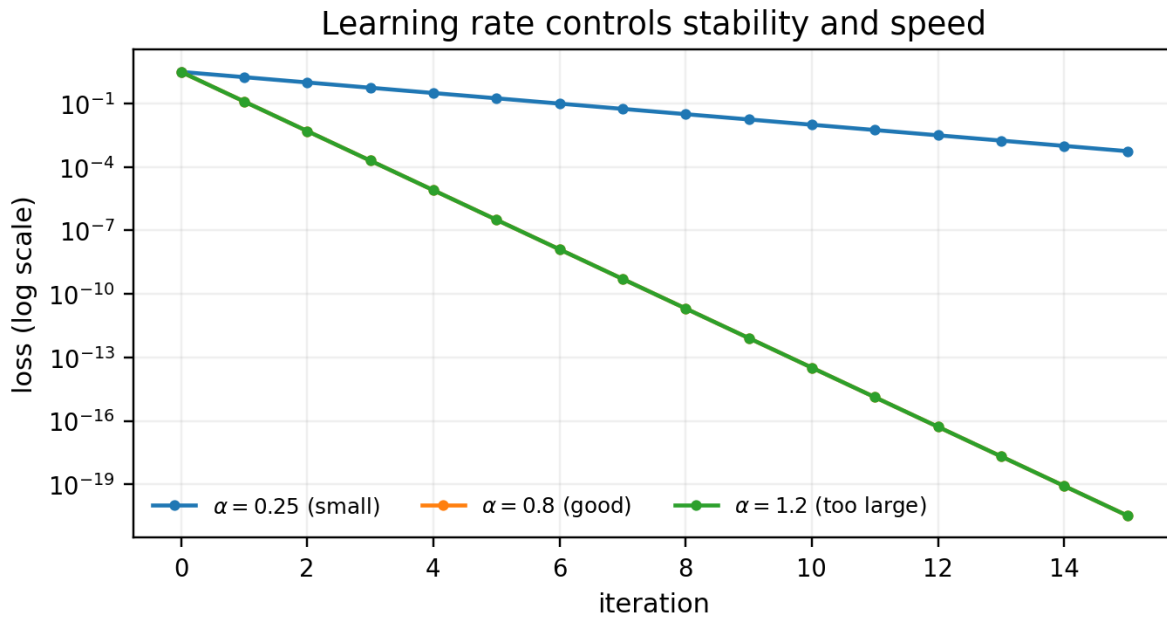


Figure 4: Illustrative behavior under different learning rates: too small, useful, oscillatory, or divergent.

6.1 Too small

If η is tiny, each update is safe but progress can be painfully slow. The model may spend many iterations making microscopic changes.

6.2 Reasonable

A suitable learning rate allows the optimizer to make substantial progress while still respecting the local geometry of the loss.

6.3 Too large

A large step can cross a valley, land on the other side, and produce an even larger loss. Repeating this can create oscillation or divergence.

6.4 A precise one-dimensional stability result

Consider the quadratic

$$f(x) = \frac{1}{2}ax^2, \quad a > 0.$$

Then

$$f'(x) = ax.$$

Gradient descent becomes

$$x_{t+1} = x_t - \eta ax_t = (1 - \eta a)x_t.$$

For convergence to 0, we need

$$|1 - \eta a| < 1,$$

which gives

$$0 < \eta < \frac{2}{a}.$$

This tiny example explains a major principle: the steeper the curvature, the smaller the stable learning rate can be.

Quadratic example:

$$f(x) = \frac{a}{2}x^2, \quad x_{t+1} = x_t - \eta a x_t = (1 - \eta a)x_t.$$

Hence convergence requires

$$|1 - \eta a| < 1 \quad \Longleftrightarrow \quad 0 < \eta < \frac{2}{a}.$$

6.5 Learning-rate schedules

In practical training, η is often changed over time. A simple step decay is

$$\eta_t = \eta_0 \gamma^{\lfloor t/k \rfloor}, \quad 0 < \gamma < 1.$$

Another common strategy is cosine decay. The exact schedule varies, but the conceptual role is consistent: large early steps promote rapid movement; smaller later steps permit finer adjustment.

In modern deep learning, the learning rate often matters as much as or more than choosing between two sophisticated optimizers.

7 Batch Gradient Descent, SGD, and Minibatches

The phrase *gradient descent* hides an important implementation choice: how much of the dataset is used to estimate each gradient?

7.1 Full-batch gradient descent

The exact empirical-risk gradient is

$$\nabla J(\theta) = \frac{1}{n} \sum_{i=1}^n \nabla \ell_i(\theta).$$

A full-batch update uses every training example before changing the parameters:

$$\theta_{t+1} = \theta_t - \eta \frac{1}{n} \sum_{i=1}^n \nabla \ell_i(\theta_t).$$

This gives a low-noise estimate of the training gradient, but it can be computationally expensive for large datasets.

7.2 Stochastic gradient descent

Pure SGD uses one randomly sampled example i_t :

$$\theta_{t+1} = \theta_t - \eta \nabla \ell_{i_t}(\theta_t).$$

The update is noisy, because one example is only an estimate of the dataset-wide direction. But that noise can be useful: it makes updates cheap and can help exploration.

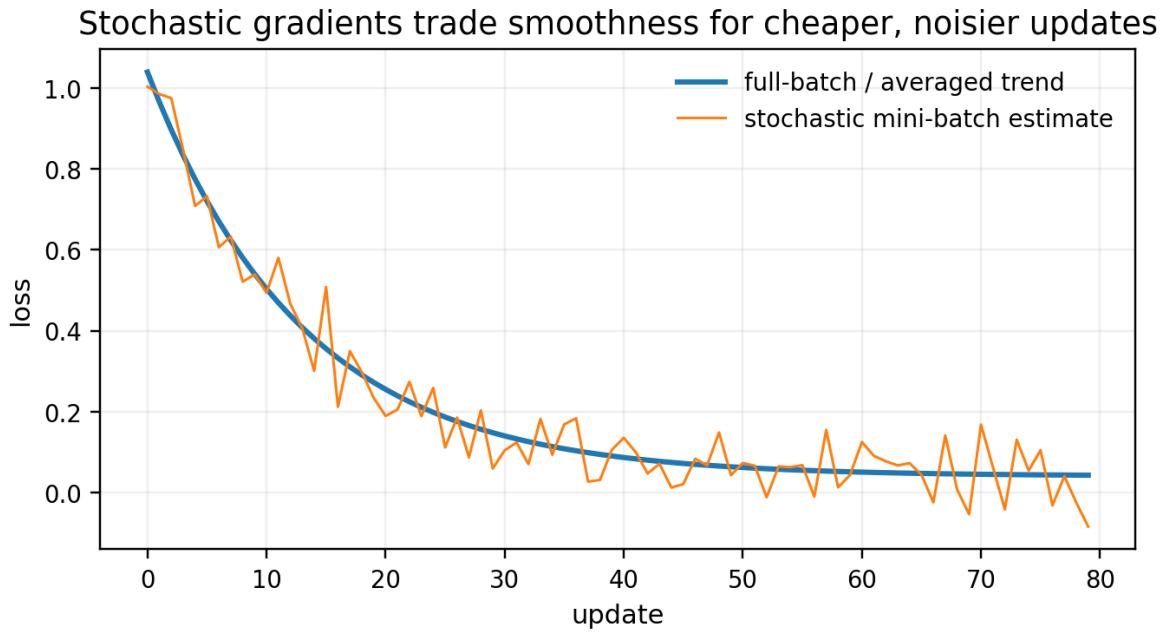


Figure 5: Stochastic gradients are noisy estimates of the full gradient. Their average direction can still point toward lower population or empirical risk.

7.3 Minibatch SGD

Modern deep learning most commonly uses a minibatch B of size m :

$$g_t = \frac{1}{m} \sum_{i \in B_t} \nabla \ell_i(\theta_t),$$

followed by

$$\theta_{t+1} = \theta_t - \eta g_t.$$

This is a compromise between the low noise of full-batch training and the low cost of single-example SGD.

7.4 Epochs, iterations, and batch size

An **iteration** usually means one parameter update. An **epoch** means one pass through the training dataset. If there are n training examples and batch size m , then approximately

$$\text{iterations per epoch} = \left\lceil \frac{n}{m} \right\rceil.$$

Therefore increasing batch size does not automatically mean fewer epochs; it means fewer or more updates per epoch depending on the definition of the training schedule.

7.5 Why minibatch gradients are useful estimators

For a uniformly sampled minibatch, the gradient estimate is approximately unbiased:

$$\mathbb{E}[g_t \mid \theta_t] = \nabla J(\theta_t).$$

Its variance generally decreases as batch size increases under standard sampling assumptions. Thus batch size controls an important tradeoff: compute efficiency, memory use, gradient noise, and number of updates.

8 A Worked Machine-Learning Example: Linear Regression

Linear regression is the cleanest example because the model, loss, gradient, and updates can all be written explicitly.

8.1 Model

For one feature x , predict

$$\hat{y} = wx + b.$$

For d features,

$$\hat{y} = \mathbf{w}^\top \mathbf{x} + b.$$

8.2 Mean squared error

For n observations:

$$J(w, b) = \frac{1}{2n} \sum_{i=1}^n (wx_i + b - y_i)^2.$$

Define residuals

$$r_i = wx_i + b - y_i.$$

Then

$$\frac{\partial J}{\partial w} = \frac{1}{n} \sum_{i=1}^n r_i x_i, \quad \frac{\partial J}{\partial b} = \frac{1}{n} \sum_{i=1}^n r_i.$$

The update is therefore

$$\begin{aligned} w &\leftarrow w - \eta \frac{1}{n} \sum_i r_i x_i, \\ b &\leftarrow b - \eta \frac{1}{n} \sum_i r_i. \end{aligned}$$

The important point is that the optimizer is not guessing w and b . The residuals influence the derivatives, and the derivatives tell us how changing w or b affects average squared error.

Linear-regression update:

$$\frac{\partial J}{\partial w} = \frac{1}{n} \sum_{i=1}^n r_i x_i, \quad \frac{\partial J}{\partial b} = \frac{1}{n} \sum_{i=1}^n r_i,$$

where $r_i = wx_i + b - y_i$. Thus

$$w \leftarrow w - \eta \frac{1}{n} \sum_i r_i x_i, \quad b \leftarrow b - \eta \frac{1}{n} \sum_i r_i.$$

8.3 Matrix form

Let $X \in \mathbb{R}^{n \times d}$ be the feature matrix, $\mathbf{w} \in \mathbb{R}^d$, and $\mathbf{y} \in \mathbb{R}^n$. Ignoring the intercept for simplicity,

$$J(\mathbf{w}) = \frac{1}{2n} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2.$$

Then

$$\nabla J(\mathbf{w}) = \frac{1}{n} X^\top (\mathbf{X}\mathbf{w} - \mathbf{y}).$$

Hence

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \frac{\eta}{n} X^\top (X \mathbf{w}_t - \mathbf{y}).$$

This is already machine learning: prediction, loss, derivative, update.

8.4 Why feature scaling matters

Suppose one feature ranges from 0 to 1 while another ranges from 0 to 1,000,000. The second feature can create much larger gradient components merely because of scale. The resulting geometry can become badly conditioned.

A common preprocessing step is standardization:

$$x'_j = \frac{x_j - \mu_j}{s_j}.$$

Scaling does not make the model smarter; it makes the optimization landscape easier to traverse.

Feature scaling changes the geometry of the optimization problem

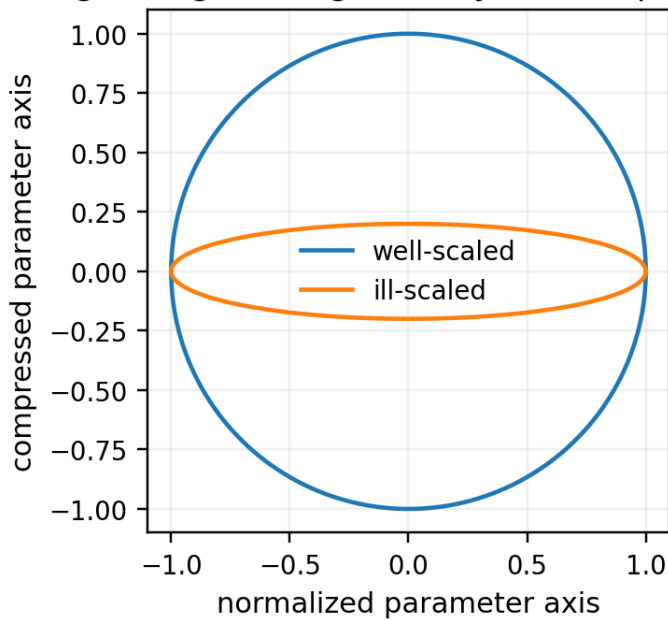


Figure 6: Ill-conditioning produces elongated valleys and inefficient zig-zagging. Scaling features can improve the numerical geometry seen by the optimizer.

9 A Second Worked Example: Logistic Regression and Cross-Entropy

Logistic regression shows why loss functions are designed together with the model's output representation.

9.1 Model

For binary classification, define a linear score

$$z = \mathbf{w}^\top \mathbf{x} + b$$

and map it to a probability with the sigmoid function:

$$p = \sigma(z) = \frac{1}{1 + e^{-z}}.$$

Logistic regression model:

$$z = \mathbf{w}^\top \mathbf{x} + b, \quad p = \sigma(z) = \frac{1}{1 + e^{-z}}.$$

9.2 Cross-entropy loss

For label $y \in \{0, 1\}$:

$$\ell(p, y) = -y \log p - (1 - y) \log(1 - p).$$

The total objective is

$$J(\mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^n \ell(p_i, y_i).$$

The particularly useful result is

$$\frac{\partial \ell}{\partial z} = p - y.$$

Therefore

$$\nabla_{\mathbf{w}} J = \frac{1}{n} \sum_{i=1}^n (p_i - y_i) \mathbf{x}_i,$$

and

$$\frac{\partial J}{\partial b} = \frac{1}{n} \sum_{i=1}^n (p_i - y_i).$$

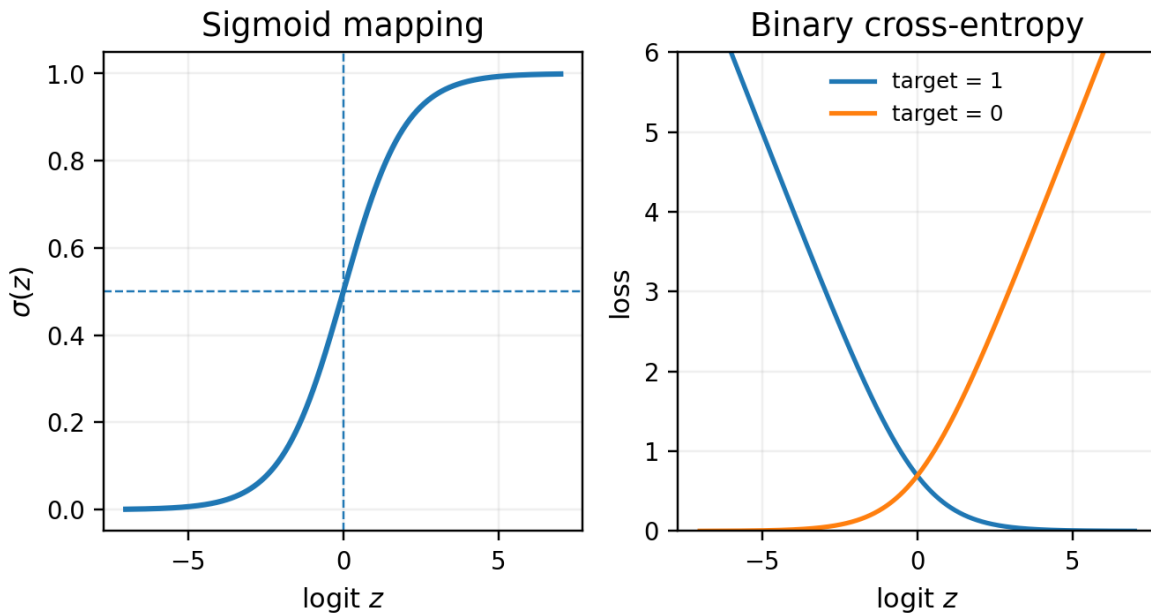


Figure 7: Cross-entropy creates a smooth objective whose gradient communicates both the direction and magnitude of classification error.

9.3 Interpretation

If the target is $y = 1$ and the model predicts $p = 0.99$, then $p - y = -0.01$, a small correction. If it predicts $p = 0.10$, then $p - y = -0.90$, a much stronger correction. The gradient naturally reflects confidence and error.

This pattern reappears in neural networks: the final loss determines the signal that backpropagation sends backward through the model.

10 Second-Order Intuition: Curvature and Why the Landscape Matters

First-order gradient descent uses the local slope. Curvature tells us how rapidly the slope itself changes.

10.1 The Hessian

For a scalar objective $J(\theta)$, the Hessian is the matrix of second derivatives:

$$H(\theta) = \nabla^2 J(\theta).$$

Its entries are

$$H_{jk} = \frac{\partial^2 J}{\partial \theta_j \partial \theta_k}.$$

The second-order Taylor expansion is

$$J(\theta + \Delta) \approx J(\theta) + \nabla J(\theta)^\top \Delta + \frac{1}{2} \Delta^\top H(\theta) \Delta.$$

10.2 Why the Hessian is conceptually useful even when we do not compute it

The Hessian describes curvature in different directions. Positive curvature means the surface bends upward in that direction; negative curvature means it bends downward. A neural-network loss can be highly anisotropic, meaning that different directions behave very differently.

Computing or storing a full Hessian for a model with millions of parameters is usually impractical: an $p \times p$ matrix has p^2 entries. First-order methods avoid this cost.

10.3 Smoothness and a useful descent inequality

Suppose the gradient is L -Lipschitz, meaning

$$\|\nabla J(\theta) - \nabla J(\phi)\| \leq L\|\theta - \phi\|.$$

Then one can bound the objective by

$$J(\theta + \Delta) \leq J(\theta) + \nabla J(\theta)^\top \Delta + \frac{L}{2} \|\Delta\|^2.$$

Substituting $\Delta = -\eta \nabla J(\theta)$ yields

$$J(\theta_{t+1}) \leq J(\theta_t) - \eta \left(1 - \frac{L\eta}{2}\right) \|\nabla J(\theta_t)\|^2.$$

Thus a sufficient condition for a guaranteed local decrease is

$$0 < \eta < \frac{2}{L}.$$

A sufficient descent condition for an L -smooth objective:

$$J(\theta_{t+1}) \leq J(\theta_t) - \eta \left(1 - \frac{L\eta}{2}\right) \|\nabla J(\theta_t)\|^2.$$

Therefore,

$$0 < \eta < \frac{2}{L}$$

is sufficient for the right-hand decrease term to be positive.

This is an important upgrade from the informal statement “pick a learning rate that works.” The stable scale is connected to the geometry of the objective.

11 Momentum and Adaptive Optimization

Plain gradient descent treats each update as if only the current gradient matters. Momentum adds memory of previous updates.

11.1 Momentum

One common form is

$$v_t = \beta v_{t-1} + g_t,$$

$$\theta_{t+1} = \theta_t - \eta v_t.$$

Here g_t is the current gradient estimate and $\beta \in [0, 1)$ controls memory.

Momentum:

$$v_t = \beta v_{t-1} + g_t, \quad \theta_{t+1} = \theta_t - \eta v_t.$$

The intuition is similar to a heavy ball rolling downhill. Consistent gradients accumulate, while rapidly alternating gradients can partially cancel. This can reduce zig-zagging in elongated valleys.

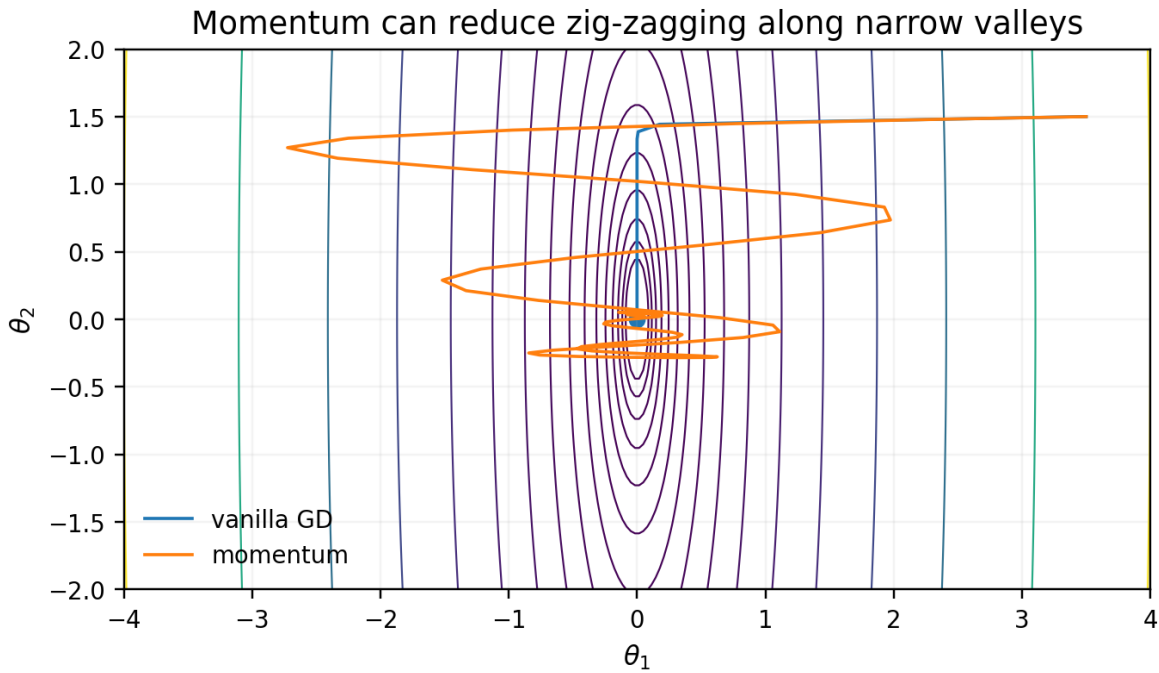


Figure 8: Momentum can produce a smoother and more directed path through an ill-conditioned region.

11.2 Nesterov momentum

Nesterov accelerated gradient modifies where the gradient is evaluated. A common expression is

$$v_t = \beta v_{t-1} + \nabla J(\theta_t - \eta \beta v_{t-1}),$$

$$\theta_{t+1} = \theta_t - \eta v_t.$$

The exact convention varies across libraries, but the central idea is to incorporate a look-ahead position when estimating the correction.

11.3 AdaGrad

AdaGrad accumulates squared gradients:

$$r_t = r_{t-1} + g_t \odot g_t,$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{r_t} + \epsilon} \odot g_t.$$

Coordinates that repeatedly receive large gradients get smaller effective step sizes.

AdaGrad:

$$r_t = r_{t-1} + g_t \odot g_t, \quad \theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{r_t} + \epsilon} \odot g_t.$$

11.4 RMSProp

RMSProp replaces the ever-growing sum with an exponential moving average:

$$r_t = \rho r_{t-1} + (1 - \rho) g_t \odot g_t,$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{r_t} + \epsilon} \odot g_t.$$

This prevents the denominator from increasing without bound.

11.5 Adam

Adam combines momentum-like first-moment tracking with second-moment tracking:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t,$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2.$$

Because these moving averages start at zero, early estimates are biased toward zero. Bias correction gives

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}.$$

The Adam update is

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}.$$

Adam:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t \odot g_t,$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t},$$

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}.$$

11.6 Do adaptive optimizers replace understanding learning rates?

No. Adam changes the effective step per parameter, but the base learning rate η remains important. Optimizer, learning rate, batch size, weight decay, schedule, initialization, and data preprocessing interact.

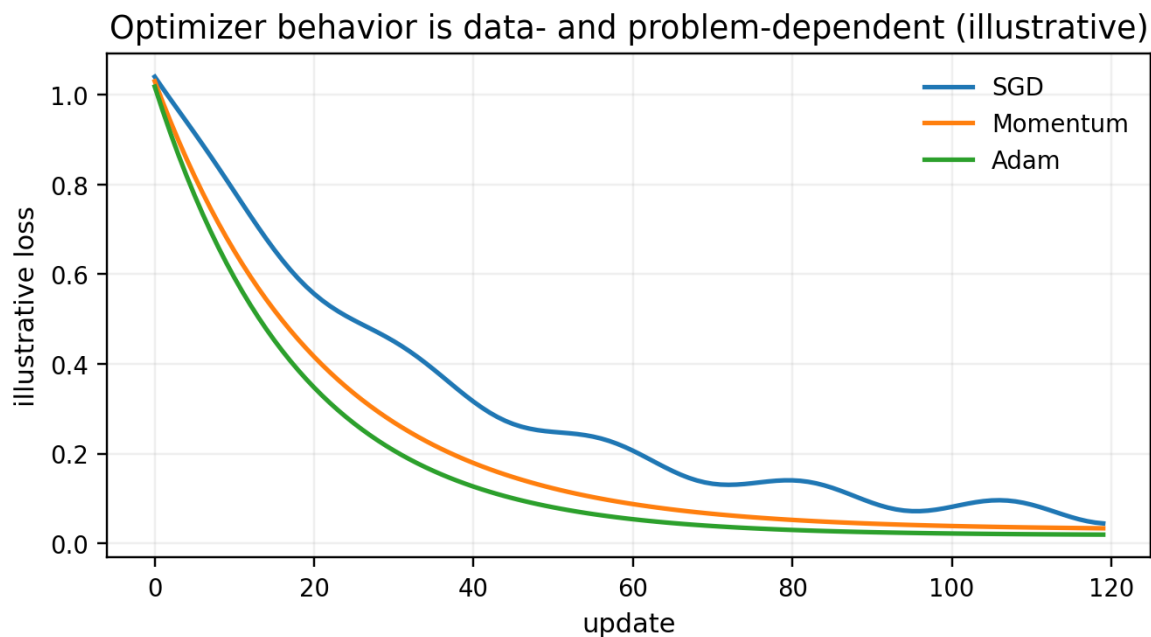


Figure 9: Different optimizers can trace visibly different trajectories even on the same objective.

12 Deep Learning: Where Gradient Descent Meets Backpropagation

This is the section that turns the calculus into deep learning.

12.1 A neural network is a parameterized computation graph

Consider a two-layer network:

$$\begin{aligned}
 \mathbf{z}^{(1)} &= W^{(1)}\mathbf{x} + \mathbf{b}^{(1)}, \\
 \mathbf{a}^{(1)} &= \phi(\mathbf{z}^{(1)}), \\
 \mathbf{z}^{(2)} &= W^{(2)}\mathbf{a}^{(1)} + \mathbf{b}^{(2)}, \\
 \hat{\mathbf{y}} &= \psi(\mathbf{z}^{(2)}).
 \end{aligned}$$

The parameters are

$$\theta = \{W^{(1)}, \mathbf{b}^{(1)}, W^{(2)}, \mathbf{b}^{(2)}\}.$$

Training means finding parameter values that minimize an objective such as

$$J(\theta) = \frac{1}{n} \sum_i \ell(f_{\theta}(x_i), y_i).$$

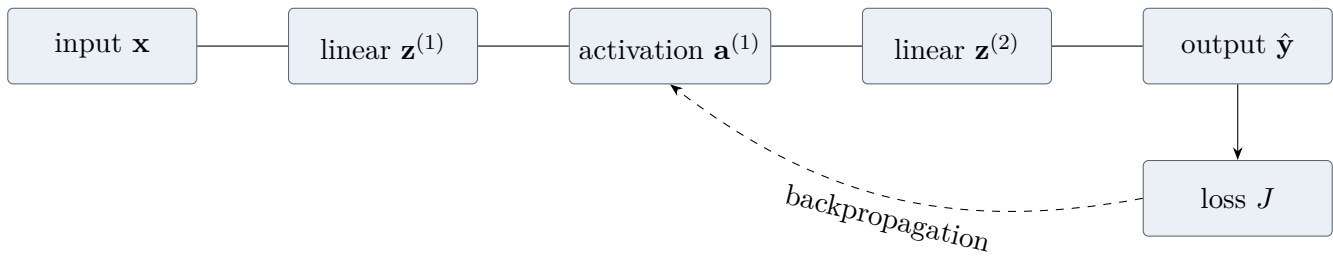


Figure 10: A neural network is a chain of differentiable operations. Backpropagation applies the chain rule backward through this computation graph.

12.2 The difficult part is not the update rule

The optimizer itself only needs the gradient. The computationally difficult question is: *how do we efficiently calculate the gradient of the loss with respect to millions of parameters?*

Backpropagation solves this using the chain rule and reusing intermediate quantities instead of differentiating each parameter independently from scratch.

12.3 The chain rule

If

$$y = f(u), \quad u = g(x),$$

then

$$\frac{dy}{dx} = \frac{dy}{du} \frac{du}{dx}.$$

For a network, the same principle is applied repeatedly through layers.

For one hidden layer, define

$$\delta^{(2)} = \frac{\partial J}{\partial \mathbf{z}^{(2)}}.$$

Then

$$\begin{aligned} \frac{\partial J}{\partial W^{(2)}} &= \delta^{(2)} (\mathbf{a}^{(1)})^\top, \\ \frac{\partial J}{\partial \mathbf{b}^{(2)}} &= \delta^{(2)}. \end{aligned}$$

The hidden-layer error signal is

$$\delta^{(1)} = \left((W^{(2)})^\top \delta^{(2)} \right) \odot \phi'(\mathbf{z}^{(1)}),$$

so

$$\begin{aligned} \frac{\partial J}{\partial W^{(1)}} &= \delta^{(1)} \mathbf{x}^\top, \\ \frac{\partial J}{\partial \mathbf{b}^{(1)}} &= \delta^{(1)}. \end{aligned}$$

Backpropagation through one hidden layer:

$$\begin{aligned} \delta^{(1)} &= \left((W^{(2)})^\top \delta^{(2)} \right) \odot \phi'(\mathbf{z}^{(1)}), \\ \frac{\partial J}{\partial W^{(1)}} &= \delta^{(1)} \mathbf{x}^\top, \quad \frac{\partial J}{\partial \mathbf{b}^{(1)}} = \delta^{(1)}. \end{aligned}$$

12.4 Backpropagation and gradient descent are different jobs

This distinction is crucial:

Backpropagation computes gradients.
Gradient descent or Adam uses those gradients to update parameters.

A deep-learning training step therefore has two conceptually separate stages:

forward pass $\rightarrow J \rightarrow$ backward pass $\rightarrow \nabla_{\theta} J \rightarrow$ optimizer step.

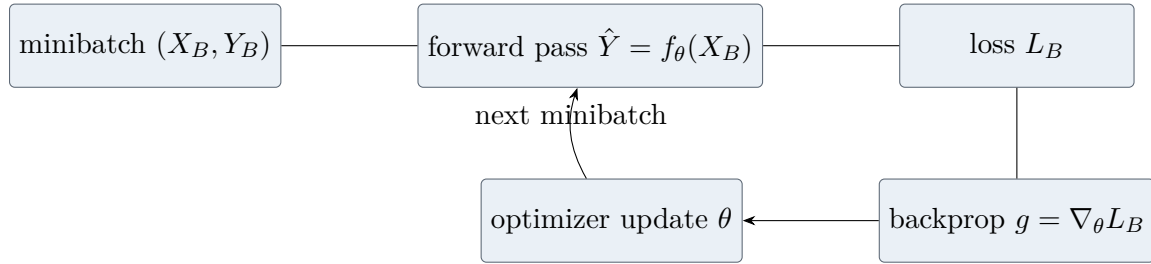


Figure 11: A typical deep-learning training loop: forward prediction, loss evaluation, backpropagation, and parameter update repeated over minibatches.

13 Deep Learning Training in More Detail

13.1 One minibatch, end to end

For minibatch B_t , the conceptual sequence is:

1. Load a batch of inputs and targets.
2. Run the network forward to obtain predictions.
3. Evaluate the batch loss.
4. Run backpropagation to compute gradients for every trainable parameter.
5. Use the optimizer to update the parameters.
6. Clear or overwrite gradients and process the next batch.

Mathematically, for parameters θ_t ,

$$\begin{aligned}
 \hat{y}_i &= f_{\theta_t}(x_i), & i &\in B_t, \\
 J_{B_t}(\theta_t) &= \frac{1}{|B_t|} \sum_{i \in B_t} \ell(\hat{y}_i, y_i), \\
 g_t &= \nabla_{\theta} J_{B_t}(\theta_t), \\
 \theta_{t+1} &= \text{Optimizer}(\theta_t, g_t).
 \end{aligned}$$

For plain SGD,

$$\theta_{t+1} = \theta_t - \eta g_t.$$

13.2 Why deep networks need nonlinearities

If every layer were linear,

$$W^{(2)}W^{(1)}x$$

would still be a linear transformation. Stacking linear layers would not create the expressive depth usually associated with neural networks. Nonlinear activations such as ReLU,

$$\phi(z) = \max(0, z),$$

allow compositions of layers to represent complex functions.

But activations also affect gradient flow. ReLU has derivative

$$\phi'(z) = \begin{cases} 1, & z > 0, \\ 0, & z < 0. \end{cases}$$

A unit that spends a long time in the zero-gradient region can stop receiving useful updates.

13.3 Vanishing and exploding gradients

Backpropagation repeatedly multiplies local Jacobians. In a simplified scalar chain,

$$\frac{\partial J}{\partial x_0} = \frac{\partial J}{\partial x_L} \prod_{l=1}^L \frac{\partial x_l}{\partial x_{l-1}}.$$

If the typical magnitudes are below 1, repeated multiplication can shrink the gradient toward zero. If they are above 1, it can grow very large. This phenomenon helped motivate careful initialization, normalization, architectural choices, residual connections, and specialized optimization practices.

13.4 Why stochasticity can help

Deep-learning training is not simply a deterministic march toward a unique answer. A minibatch gradient contains sampling noise. The resulting update can be viewed loosely as

$$g_t = \nabla J(\theta_t) + \varepsilon_t,$$

where ε_t is gradient-estimation noise.

That noise can make the optimization path less predictable than full-batch gradient descent. In practice, stochasticity can sometimes help models move away from narrow regions and may influence which solutions are reached, though the exact behavior depends on architecture, data, optimizer, schedule, and scale.

13.5 Training loss versus validation loss

The optimizer minimizes the objective presented on training batches. It does not directly optimize validation performance. Therefore a training curve falling steadily does not imply that generalization is improving indefinitely.

A useful mental picture is:

$$\text{optimization} \neq \text{generalization}.$$

Optimization asks whether we can reduce the chosen training objective. Generalization asks whether the resulting function performs well on new samples.

14 How Gradient Descent Appears Across Machine Learning

Gradient descent is important because it is not a neural-network-only trick.

14.1 Linear regression

For differentiable squared error:

$$\min_{\mathbf{w}} \frac{1}{2n} \|X\mathbf{w} - \mathbf{y}\|^2.$$

Gradient methods can solve the problem iteratively, although a closed-form solution also exists under suitable assumptions.

14.2 Logistic regression

For cross-entropy:

$$\min_{\mathbf{w}, b} -\frac{1}{n} \sum_i [y_i \log p_i + (1 - y_i) \log(1 - p_i)].$$

Gradient methods are a standard way to fit the parameters.

14.3 Generalized linear models

Many generalized linear models use differentiable likelihood-based objectives, making gradient-based optimization natural.

14.4 Matrix factorization and recommender systems

A factorization model may represent a rating matrix approximately as

$$R \approx UV^T.$$

A squared reconstruction objective might be

$$J(U, V) = \sum_{(i,j) \in \Omega} (R_{ij} - U_i V_j^T)^2.$$

Alternating or joint gradient-based methods can optimize these latent factors.

14.5 Neural networks

For a neural network,

$$\min_{\theta} J(\theta)$$

is generally a nonlinear, high-dimensional problem with no useful global closed-form solution. Gradient-based methods are therefore central.

14.6 Support vector machines

The classical soft-margin SVM objective can be optimized with methods that exploit gradients or subgradients. For example, hinge loss is

$$\ell(y, f(x)) = \max(0, 1 - yf(x)).$$

The hinge loss is not differentiable at the kink, so a subgradient can be used.

14.7 Gradient boosting

Gradient boosting is related but conceptually different. It performs functional optimization by fitting new weak learners to negative gradients of the loss with respect to current predictions. It is therefore often described as gradient descent in function space rather than ordinary parameter-space gradient descent.

If the current predictor is $F_{m-1}(x)$, the negative pseudo-residuals are conceptually

$$r_i^{(m)} = - \left. \frac{\partial \ell(y_i, F(x_i))}{\partial F(x_i)} \right|_{F=F_{m-1}}.$$

A new learner is fit to these residuals, then added with a shrinkage factor.

This is a particularly useful connection to know: “gradient descent” does not have to mean adjusting a vector of neural-network weights. It is a general idea of moving an object in the direction that decreases an objective.

15 What Machine-Learning Methods Do Not Need Gradient Descent?

Not every machine-learning algorithm is naturally trained with gradient descent.

Method	Main training idea	Gradient descent role
Linear regression	Closed-form least squares or iterative optimization	Optional for basic form
Logistic regression	Likelihood/cross-entropy optimization	Commonly used
Decision tree	Greedy split selection	Not required
Random forest	Bagged decision trees	Not required
Naive Bayes	Estimate probabilities/parameters	Not required in standard form
k -means	Alternate assignments and centroids	Not required in standard algorithm
SVM	Convex optimization / subgradients	Often gradient-related, but not identical to neural-net training
Neural network	Nonlinear parameter optimization	Central
Gradient boosting	Sequential functional optimization	Gradient concept is central, implementation varies

The lesson is not “every ML model uses gradient descent.” The lesson is that many important differentiable learning problems can be expressed as optimization problems, and gradient-based methods are one of the most scalable ways to solve them.

15.1 Why trees behave differently

A decision tree chooses splits such as

$$x_j < c$$

that improve a criterion such as impurity reduction. The candidate split threshold is discrete or piecewise-defined, so the problem does not resemble smoothly tuning continuous weights.

This is why random forests can be trained without backpropagation or gradient descent even though they are powerful machine-learning models.

16 Regularization, Weight Decay, and the Objective You Actually Optimize

The practical objective is often more than prediction error.

16.1 L2 regularization

Suppose we add a penalty:

$$J_{\text{reg}}(\theta) = J_{\text{data}}(\theta) + \frac{\lambda}{2} \|\theta\|_2^2.$$

Then

$$\nabla J_{\text{reg}}(\theta) = \nabla J_{\text{data}}(\theta) + \lambda\theta.$$

Plain gradient descent gives

$$\theta_{t+1} = \theta_t - \eta (\nabla J_{\text{data}}(\theta_t) + \lambda\theta_t).$$

Rearranging,

$$\theta_{t+1} = (1 - \eta\lambda)\theta_t - \eta\nabla J_{\text{data}}(\theta_t).$$

This reveals the “weight decay” intuition: in addition to moving according to data loss, the update tends to shrink parameters.

16.2 L1 regularization

With

$$J_{\text{reg}}(\theta) = J(\theta) + \lambda\|\theta\|_1,$$

we encounter the absolute value function, whose derivative is not defined at zero. Optimization can use subgradients or proximal methods. L1 penalties are associated with sparse solutions under appropriate conditions.

16.3 The practical lesson

When you hear that an optimizer “minimizes the loss,” mentally check what loss is actually being optimized. Training objectives can include regularization terms, auxiliary losses, or task-specific penalties.

17 Convergence, Minima, Saddles, and Nonconvexity

The word “minimum” needs care.

17.1 Convex objectives

An objective J is convex when, informally, the line segment between any two points lies above the graph:

$$J(\alpha\theta + (1 - \alpha)\phi) \leq \alpha J(\theta) + (1 - \alpha)J(\phi), \quad \alpha \in [0, 1].$$

For differentiable convex problems, any local minimum is global. This makes optimization theory much cleaner.

Strong convexity adds a quadratic lower bound and yields stronger convergence guarantees. For a smooth strongly convex objective, appropriately chosen gradient descent converges at a linear rate.

17.2 Neural networks are typically nonconvex

Deep-network objectives are generally nonconvex in their parameters. The landscape can contain many critical points, flat directions, symmetries, and saddle points.

A critical point satisfies

$$\nabla J(\theta) = 0.$$

But $\nabla J = 0$ does not imply a minimum. It can be a maximum or saddle point.

For a twice-differentiable function, the Hessian helps classify a critical point. At a local minimum, the Hessian is positive semidefinite under standard second-order conditions. A saddle can have both positive and negative curvature directions.

17.3 Why “gradient descent finds the global minimum” is the wrong generic statement

For arbitrary nonconvex objectives, there is no universal guarantee that vanilla gradient descent reaches the global minimum. What is more accurate is that under suitable conditions, gradient-based methods can converge to stationary points or achieve useful practical solutions, and the behavior depends strongly on the objective structure and stochasticity.

This distinction matters when discussing deep learning. Neural network training works remarkably well without reducing the problem to a simple convex bowl.

18 Optimization Pathologies in Deep Learning

18.1 Vanishing gradients

A gradient can become tiny as it is propagated backward through many operations. When the updates become effectively zero,

$$\theta_{t+1} \approx \theta_t,$$

learning stalls for the affected parameters.

18.2 Exploding gradients

The opposite problem occurs when gradients become extremely large:

$$\|\nabla J(\theta_t)\| \gg 1.$$

Updates can become numerically unstable. Gradient clipping is a practical intervention, often taking the form

$$g \leftarrow g \min \left(1, \frac{c}{\|g\|} \right),$$

where c is a clipping threshold.

18.3 Ill-conditioning

When some directions are much steeper than others, one learning rate has to satisfy conflicting requirements. It must be small enough not to overshoot steep directions but large enough to make progress along shallow ones.

Momentum and adaptive scaling can help, while feature normalization, architectural design, and appropriate parameterization can improve the underlying geometry.

18.4 Saturation

Sigmoid and tanh activations can have derivatives close to zero in saturated regions. For sigmoid,

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)),$$

which has a maximum of 1/4 and approaches zero as $|z|$ grows large.

18.5 Dead units

For ReLU, a unit with consistently negative preactivation can have derivative zero. The corresponding path may stop receiving gradient updates from those examples.

18.6 Numerical issues

Exponentials and logarithms can create overflow or underflow. For example, evaluating softmax naively can overflow for large logits. Stable implementations subtract the maximum logit:

$$p_k = \frac{e^{z_k - \max_j z_j}}{\sum_j e^{z_j - \max_j z_j}}.$$

This leaves the mathematical probabilities unchanged while improving numerical stability.

19 What the Training Curves Are Telling You

Learning curves are among the most useful practical views into optimization.

A typical plot might contain training loss and validation loss against iteration or epoch.

- **Both decrease:** optimization and generalization are improving.
- **Training decreases, validation increases:** overfitting is a likely explanation.
- **Both barely move:** the learning rate may be too small, gradients may be ineffective, or the model/data pipeline may have another issue.
- **Loss oscillates sharply:** the learning rate may be too large, gradients may be noisy, or the objective may be poorly conditioned.
- **Loss becomes NaN:** investigate exploding values, unstable operations, invalid data, or numerical overflow.

A useful debugging habit is to separate the chain:

data $\rightarrow$ **forward pass** $\rightarrow$ **loss** $\rightarrow$ **gradients** $\rightarrow$ **optimizer update**.

If training fails, inspect the chain in that order rather than immediately changing the optimizer.

19.1 Checking gradient magnitudes

For parameter block θ_k , monitor quantities such as

$$\|\theta_k\|_2, \quad \|\nabla_{\theta_k} J\|_2, \quad \frac{\|\nabla_{\theta_k} J\|_2}{\|\theta_k\|_2 + \epsilon}.$$

These statistics can reveal layers with unusually small or large gradients.

19.2 Why a low loss is not the entire story

A model can minimize a chosen objective while producing outputs that are undesirable under a different metric. In classification, for example, cross-entropy is smooth and gradient-friendly, while the final reported metric might be accuracy, F1, calibration, or expected cost.

Optimization therefore sits inside a larger design problem: choose an objective whose gradients encourage the behavior you actually want.

20 A Practical Mental Model for Deep-Learning Optimizers

When comparing SGD, momentum, RMSProp, and Adam, avoid memorizing them as unrelated formulas. Think of them as modifications to a common pattern.

gradient estimate	→	transform gradient	→	update parameters
-------------------	---	--------------------	---	-------------------

Plain SGD:

$$g_t = \nabla J(\theta_t), \quad \Delta\theta_t = -\eta g_t.$$

Momentum:

keep memory of recent gradients or velocities.

RMSProp/AdaGrad:

scale coordinates using historical squared gradients.

Adam:

combine first-moment direction + second-moment scaling.

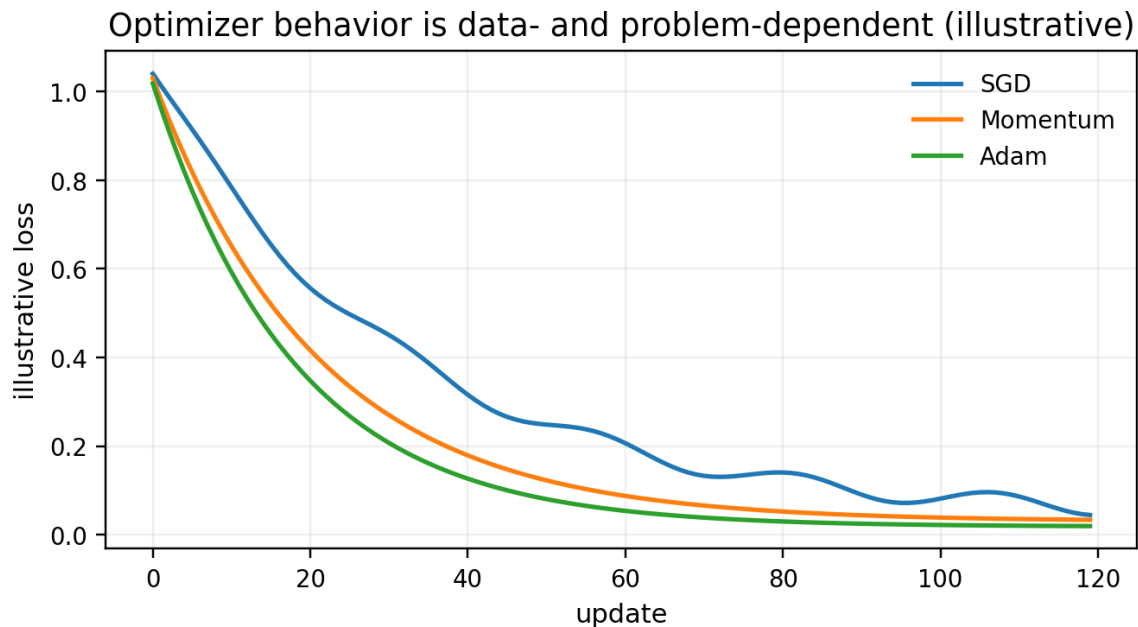


Figure 12: A conceptual comparison: optimizers differ mainly in how they transform gradient information before changing parameters.

20.1 A useful interpretation of Adam

Adam does not “understand” the problem. It has no concept of images, language, music, or semantics. It detects statistical properties of the gradient sequence and uses them to choose coordinate-wise step sizes and directions.

20.2 Why SGD is still important to understand

Even when a project uses Adam, understanding SGD is essential because Adam is easier to understand as a modification of the same core idea. If the equation

$$\theta \leftarrow \theta - \eta g$$

is intuitive, the optimizer family becomes much easier to reason about.

21 Pseudocode and the Actual Training Loop

The core algorithm is short.

Gradient descent pseudocode

```
initialize theta
repeat:
    compute predictions f_theta(x)
    compute loss J(theta)
    compute gradient g = grad_theta J(theta)
    theta = theta - eta * g
until stopping condition
```

For minibatch deep learning, a practical high-level version is:

```
initialize model parameters theta
initialize optimizer state
for epoch in range(num_epochs):
    shuffle training data
    for minibatch (x, y):
        y_hat = model(x)
        loss = criterion(y_hat, y)
        zero old gradients
        backpropagate loss
        optimizer step
```

The framework may hide the calculus, but those conceptual stages are still present.

21.1 Automatic differentiation

Modern deep-learning systems usually compute gradients using automatic differentiation. Instead of asking a programmer to hand-derive every derivative for every operation, the framework records a computational graph and applies differentiation rules systematically.

That is why it is useful to think of a network as a graph of operations. Each operation knows how to propagate an incoming derivative backward to its inputs.

21.2 One training step in symbols

For minibatch B :

$$\begin{aligned}\hat{\mathbf{y}} &= f_{\theta}(X_B), \\ L_B &= \frac{1}{|B|} \sum_{i \in B} \ell(\hat{y}_i, y_i), \\ g &= \frac{\partial L_B}{\partial \theta},\end{aligned}$$

then, for SGD,

$$\theta \leftarrow \theta - \eta g.$$

Everything else in the training loop exists to feed these steps efficiently and safely.

22 A Deeper View: Gradient Descent as Optimization in Parameter Space

There is a useful conceptual leap here. A neural network can be regarded as a point in a high-dimensional parameter space. Training is a trajectory through that space.

At iteration t ,

$$\theta_t \in \mathbb{R}^p.$$

The optimizer constructs

$$\theta_{t+1} = \theta_t + \Delta_t.$$

Gradient descent chooses

$$\Delta_t = -\eta_t g_t.$$

Adam chooses a transformed version of g_t . Momentum adds temporal state. All are choices about the trajectory through the same parameter space.

22.1 Reparameterization and effective geometry

The same model function can sometimes be represented by different parameter values. Neural networks also contain symmetries: permuting hidden units can leave the represented function unchanged. Consequently, the raw parameter landscape can contain multiple equivalent solutions.

This helps explain why asking for “the” optimal neural-network weights is often conceptually misleading. There may be many different parameter configurations that implement equally useful functions.

22.2 Why optimization and architecture interact

Architecture determines the computational graph and therefore the derivatives. Normalization layers, skip connections, activation functions, and parameterization choices all affect the gradient flow.

Therefore optimization is not merely an after-the-fact choice of Adam versus SGD. The architecture itself shapes how easy the objective is to optimize.

22.3 Why initialization matters

At initialization, the network may already be in a regime where signals and gradients are too small, too large, or badly scaled. Initialization schemes attempt to control activation and gradient magnitudes. A

simplified variance-preserving intuition is that if a neuron combines n_{in} inputs, the weight scale should decrease as fan-in grows.

He/Xavier-style initializations are examples of this general principle. They are not magic constants; they are attempts to place the network in a numerically workable regime before gradient-based learning begins.

23 The Most Important Equations to Know

This section is intended as a compact mathematical memory aid.

23.1 Objective

$$J(\theta) = \frac{1}{n} \sum_{i=1}^n \ell(f_{\theta}(x_i), y_i).$$

23.2 Gradient

$$\nabla_{\theta} J(\theta) = \begin{bmatrix} \partial J / \partial \theta_1 \\ \vdots \\ \partial J / \partial \theta_p \end{bmatrix}.$$

23.3 Gradient descent

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t).$$

23.4 Minibatch SGD

$$g_t = \frac{1}{m} \sum_{i \in B_t} \nabla \ell_i(\theta_t), \quad \theta_{t+1} = \theta_t - \eta g_t.$$

23.5 Momentum

$$v_t = \beta v_{t-1} + g_t, \quad \theta_{t+1} = \theta_t - \eta v_t.$$

23.6 RMSProp

$$r_t = \rho r_{t-1} + (1 - \rho) g_t^2, \quad \theta_{t+1} = \theta_t - \eta \frac{g_t}{\sqrt{r_t} + \epsilon}.$$

23.7 Adam

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t, \\ v_t &= \beta_2 v_{t-1} + (1 - \beta_2) g_t^2, \\ \hat{m}_t &= \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}, \end{aligned}$$

$$\theta_{t+1} = \theta_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}.$$

23.8 Regularized objective

$$J_{\text{reg}}(\theta) = J(\theta) + \frac{\lambda}{2} \|\theta\|_2^2.$$

23.9 Backpropagation idea

$$\frac{\partial J}{\partial x} = \frac{\partial J}{\partial y} \frac{\partial y}{\partial x}$$

for a differentiable composition $x \rightarrow y \rightarrow J$.

If these equations are conceptually clear rather than merely familiar-looking, you already understand the mathematical skeleton of most gradient-based deep-learning training.

24 How to Study Gradient Descent So It Actually Sticks

A useful learning order is not “memorize optimizers.” Build the dependency chain.

1. Understand a scalar loss as a score that training wants to reduce.
2. Understand a derivative as local slope.
3. Extend derivatives to a gradient vector.
4. Understand why $-\nabla J$ is a descent direction.
5. Understand the learning rate as step size.
6. Understand minibatches as noisy gradient estimates.
7. Learn the chain rule and backpropagation.
8. Only then learn momentum, RMSProp, Adam, schedules, and weight decay.

24.1 The mental translation to use when reading papers or code

When you see

```
loss.backward()
```

mentally translate it to: *compute $\nabla_{\theta} J$ using the chain rule through the computational graph.*

When you see

```
optimizer.step()
```

translate it to: *use those gradients, plus any optimizer state, to produce a new θ .*

When you see

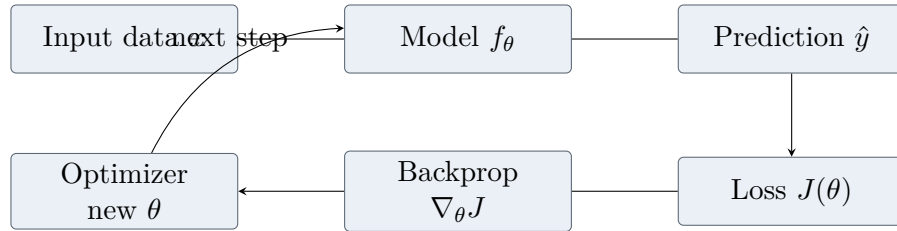
```
lr=1e-3
```

translate it to: *the optimizer’s base step scale is 10^{-3} , subject to its update rule and any schedule.*

When you see “batch size 64,” translate it to: *each gradient estimate is based on 64 examples at a time, subject to the implementation.*

When you see “3 epochs,” translate it to: *the training data is iterated over three times; the number of parameter updates is determined by the batch size and data size.*

24.2 A compact conceptual map



This is the core architecture of gradient-based learning.

25 Common Misconceptions

1. “Gradient descent is the AI.”

No. It is an optimization algorithm. The model architecture, data, loss, and optimizer are separate components.

2. “The gradient is the error.”

Not exactly. The loss is an error-like scalar objective; the gradient is the vector of sensitivities of that objective with respect to parameters.

3. “A negative gradient means the loss is negative.”

No. The negative sign indicates the direction in which the parameters should move to locally decrease loss.

4. “Adam is fundamentally different from gradient descent.”

Adam is a gradient-based optimizer built around the same fundamental idea of using derivatives to update parameters, with momentum-like and adaptive scaling mechanisms.

5. “Backpropagation trains the network.”

Backpropagation computes gradients. An optimizer then uses those gradients to update parameters.

6. “Lower training loss always means a better model.”

Not necessarily. Generalization depends on the relationship between the learned function and unseen data.

7. “A larger learning rate is always better because training is faster.”

No. Increasing the step size can accelerate progress only until stability and curvature become limiting factors.

8. “If the gradient is zero, we are done.”

Not generally in a nonconvex problem. A zero gradient can occur at minima, maxima, or saddle points, and numerical near-zero gradients can also indicate a training pathology.

26 Final Synthesis

Gradient descent is easiest to understand as a remarkably simple idea operating inside a complicated machine-learning system.

A model maps inputs to predictions:

$$\hat{y} = f_\theta(x).$$

A loss turns prediction quality into a scalar:

$$J(\theta).$$

Differentiation tells us how that scalar changes when the parameters change:

$$\nabla_{\theta} J(\theta).$$

The optimizer uses that information to alter the parameters:

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t)$$

or a more elaborate variant such as Adam.

For deep learning, backpropagation is what makes the gradient computationally feasible. It reuses the structure of the network’s computation graph so the effect of the loss can be propagated backward through layers.

The complete deep-learning picture is therefore:

data $\rightarrow$ forward pass $\rightarrow$ loss $\rightarrow$ backpropagation $\rightarrow$ gradient $\rightarrow$ optimizer $\rightarrow$ new parameters

The most important conceptual distinction is that the optimizer does not “know” what it is learning. It only receives numerical derivative information. The intelligence of the system emerges from the combination of data, architecture, objective, and repeated optimization.

That is why gradient descent is so important for understanding deep learning. Once the update rule, gradient, loss, and backpropagation are clear, many apparently different training mechanisms become variations on the same mathematical theme.

The deepest takeaway: deep learning is not primarily “a neural network guessing better.” It is a parameterized function being repeatedly adjusted because calculus tells us how a chosen objective changes under tiny parameter changes. Gradient descent is the mechanism that turns that local information into learning.

References and Further Reading

1. Goodfellow, I., Bengio, Y., and Courville, A. *Deep Learning*. MIT Press, 2016. Chapters 4 and 8 are especially relevant to numerical computation and optimization.
2. Bishop, C. M. *Pattern Recognition and Machine Learning*. Springer, 2006.
3. Hastie, T., Tibshirani, R., and Friedman, J. *The Elements of Statistical Learning*. Springer, 2nd ed., 2009.
4. Ruder, S. “An overview of gradient descent optimization algorithms.” arXiv:1609.04747, 2016.
5. Kingma, D. P. and Ba, J. “Adam: A Method for Stochastic Optimization.” arXiv:1412.6980, 2014.
6. Robbins, H. and Monro, S. “A Stochastic Approximation Method.” *The Annals of Mathematical Statistics*, 1951.
7. Polyak, B. T. “Some Methods of Speeding Up the Convergence of Iteration Methods.” *USSR Computational Mathematics and Mathematical Physics*, 1964.
8. Nesterov, Y. *Introductory Lectures on Convex Optimization*. Kluwer, 2004.
9. LeCun, Y., Bengio, Y., and Hinton, G. “Deep learning.” *Nature*, 2015.

A Notation Cheat Sheet

Symbol	Meaning
x	Input/features for one example
y	Target/label
$\hat{y}$	Model prediction
θ	Vector of trainable parameters
$J(\theta)$	Training objective / loss function
ℓ	Per-example loss
$\nabla_{\theta} J$	Gradient of objective with respect to parameters
η	Learning rate
g_t	Minibatch or stochastic gradient estimate
β, β_1, β_2	Exponential-memory coefficients in momentum/Adam-style methods
λ	Regularization strength
$H = \nabla^2 J$	Hessian / matrix of second derivatives
ϵ	Small numerical-stability constant
n	Number of training examples
m	Minibatch size
t	Optimization iteration index